

Database (Trove): Trove is used as database-as-a-service (DaaS), which integrates and provisions relational and non-relational database engines.

Elastic Map Reduce (Sahara): Sahara is the specialised service that enables data processing on OpenStack-managed resources, including the processing with Apache Hadoop.

Deployment of OpenStack using DevStack

DevStack is used to quickly create an OpenStack development environment. It is also used to demonstrate the starting and running of OpenStack services, and provide examples of using them from the command line. DevStack has evolved to support a large number of configuration options and alternative platforms and support services. It can be considered as the set of scripts which install all the essential OpenStack services in the computer without any additional software or configuration. To implement DevStack, first download all the essential packages, pull in the OpenStack code from various OpenStack projects, and set everything for the deployment.

To install OpenStack using DevStack, any Linux-based distribution with 2GB RAM can be used to start the implementation of IaaS.

Here are the steps that need to be followed for the installation.

1. Install Git

```
$ sudo apt-get install git
```

2. Clone the DevStack repository and change the directory. The code will set up the cloud infrastructure.

```
$ git clone http://github.com/openstack-dev/devstack
$ cd devstack/
```

```
/devstack$ ls
```

```
accrc      exercises  HACKING.rst  rejoin-stack.sh  tests
AUTHORS    exercise.sh lib          run_tests.sh    tools
clean.sh   extras.d   LICENSE      samples          unstack.sh
driver_certs files      localrc      stackrc
eucarc     functions openrc       stack-screenrc
exerciserc functions-common README.md     stack.sh
```

stack.sh, *unstack.sh* and *rejoin-stack.sh* are the most important files. *stack.sh* script is used to set up DevStack. *unstack.sh* is used to destroy the DevStack setup.

If you are on the earlier execution of *.stack.sh*, the environment can be brought up by executing the *rejoin_stack.sh* script.

3. Execute the *stack.sh* script:

```
/devstack$ ./stack.sh
```

Here, the MySQL database password is entered. There's no need to worry about the installation of MySQL separately

on this system. We have to specify a password and this script will install MySQL, and use this password there.

Finally, we will have the script ending as follows:

```
+ merge_config_group /home/r/devstack/local.conf post-extra
+ local localfile=/home/r/devstack/local.conf
+ shift
+ local matchgroups=post-extra
+ [[ -r /home/r/devstack/local.conf ]]
+ return 0
+ [[ -x /home/r/devstack/local.sh ]]
+ service_check
+ local service
+ local failures
+ SCREEN_NAME=stack
+ SERVICE_DIR=/opt/stack/status
+ [[ ! -d /opt/stack/status/stack ]]
++ ls /opt/stack/status/stack/*.failure'
++ /bin/true
+ failures=
+ '[' -n '' ']'
+ set +o xtrace
```

- Horizon is now available at <http://1.1.1.1/>
- Keystone is serving at <http://1.1.1.1:5000/v2.0/>
- Examples on using the *novaclient* command line are in *exercise.sh*
- The default users are: admin and demo
- The password: nova
- This is your host IP: 1.1.1.1

After all these steps, the machine becomes the cloud service providing platform. Here, 1.1.1.1 is the IP of my first network interface.

We can type the host IP provided by the script into a browser, in order to access the dashboard 'Horizon'. We can log in with the username 'admin' or 'demo' and the password 'admin'.

You can view all the process logs inside the screen, by typing the following command:

```
$ screen -x
```

Executing the following will kill all the services, but it should be noted that it will not delete any of the code.

To bring down all the services manually, type:

```
$ sudo killall screen
```

localrc configurations

localrc is the file in which all the local configurations (local machine parameters) are maintained. After the first successful *stack.sh* run, you will see that a *localrc* file gets created with the configuration values you specified while running that script.

The following fields are specified in the *localrc* file: